

Система технического зрения и детектирование объектов

В этом уроке мы научимся размечать датасет для обучения нейронной сети.

Исходные изображения для датасета будут собираться из готовой сцены в симуляторе Webots, поэтому предполагается, что вы уже установили симулятор и нужные пакеты для выполнения заданий.

Для разметки датасета будет использоваться сайт Roboflow, а детектирование объектов будет производиться с помощью нейросети YOLO.

Что такое YOLO?

YOLO (You Only Look Once) - это одноэтапный (single-stage) алгоритм детекции объектов на изображении.

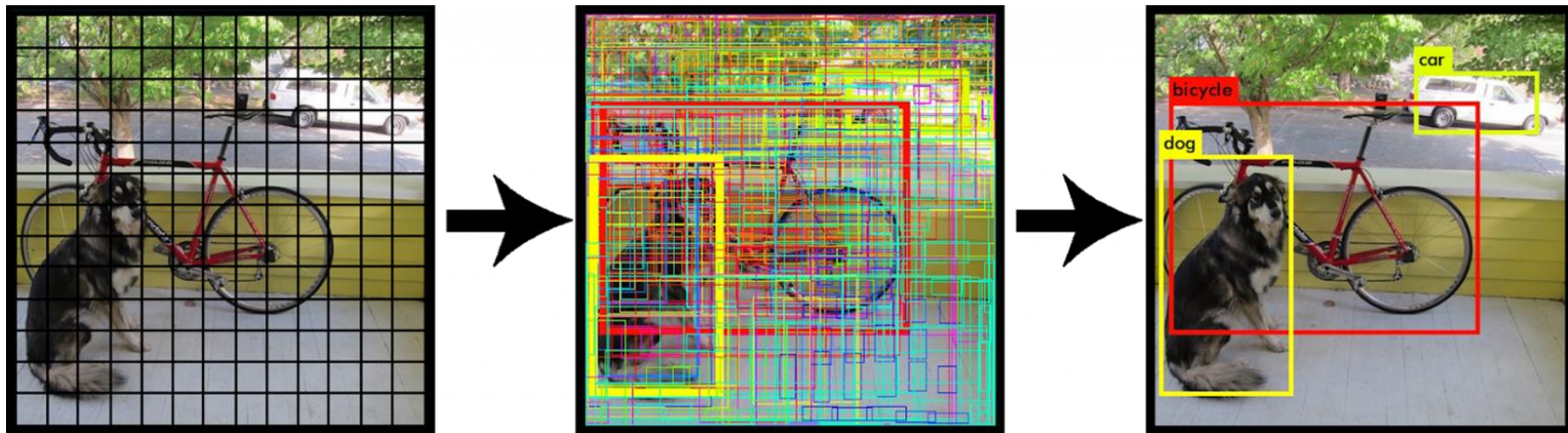
Принцип работы:

Входное изображение делится на сетку ячеек (например, $S \times S$).

Для каждой ячейки модель предсказывает:

- несколько ограничивающих рамок (bounding boxes) с координатами (x, y, w, h) и уровнем уверенности;
- вероятности принадлежности объектов к заданным классам.

Все вычисления выполняются **за один проход** сети - это и даёт выигрыш в скорости.



Зачем размечать датасет?

Разметка датасета - ключевой этап в обучении моделей компьютерного зрения (например, YOLO), поскольку она:

- **Обучает модель «понимать» объекты**

Размеченные данные показывают модели, какие объекты искать, где они расположены и к каким классам относятся. Например, для YOLO нужно указать координаты прямоугольников (bounding boxes) вокруг машин, людей, животных и т. д.

- **Повышает точность и надёжность модели**

Чёткие, качественные метки позволяют модели лучше обобщать паттерны, избегать ошибок и точнее определять объекты даже в сложных условиях (перекрытие, частичное появление, разное освещение).

- **Адаптирует модель под конкретную задачу**

Вы определяете, какие классы объектов важны для вашей задачи (например, «стоп-знак», «пешеход» для автономного транспорта), и размечаете данные соответственно. Это позволяет не обучать модель на избыточных данных.

- **Обеспечивает возможность валидации и тестирования**

Размеченные данные используются для оценки производительности модели: сравнения предсказаний с «золотым стандартом» (метками), расчёта метрик (mAP, IoU) и выявления слабых мест.

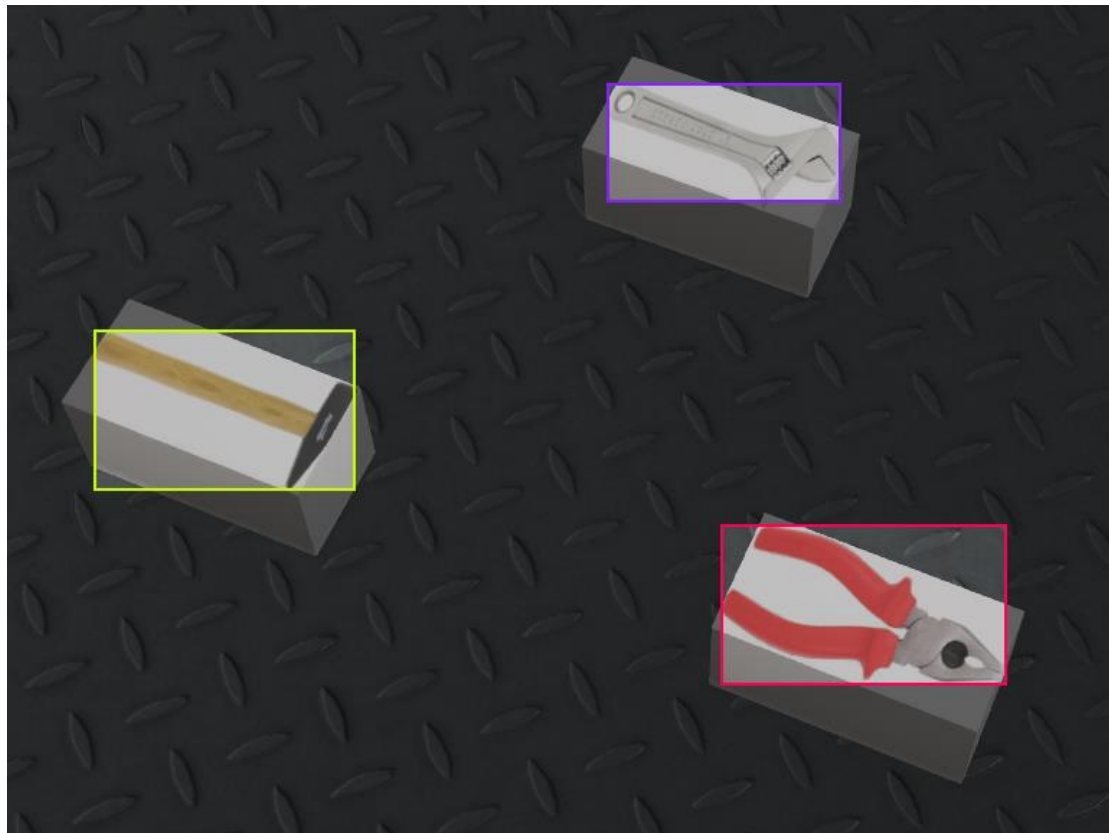
- **Стандартизирует входные данные**

YOLO требует строго определённого формата данных (например, координаты в нормализованном виде, список классов). Разметка приводит данные к единому виду, упрощая обучение и инференс.

Коротко:

Без разметки модель не сможет «увидеть» связь между изображениями и ожидаемыми результатами — разметка превращает «сырые» данные в обучающий материал, понятный для нейросети.

Пример размеченного изображения из датасета



Приступим к созданию своего датасета

Датасет будем создавать для детектирования инструментов (плоскогубцы, молоток, гаечный ключ). В качестве исходных данных будем использовать скриншоты из готовой сцены симулятора Webots.

Открываем терминал или IDE, которую вы используете и переходим в директорию, в которой лежит директория src с установленными пакетами для соревнований

```
cd ~/ros2_ws
```

Если еще не собрали пакеты - собираем

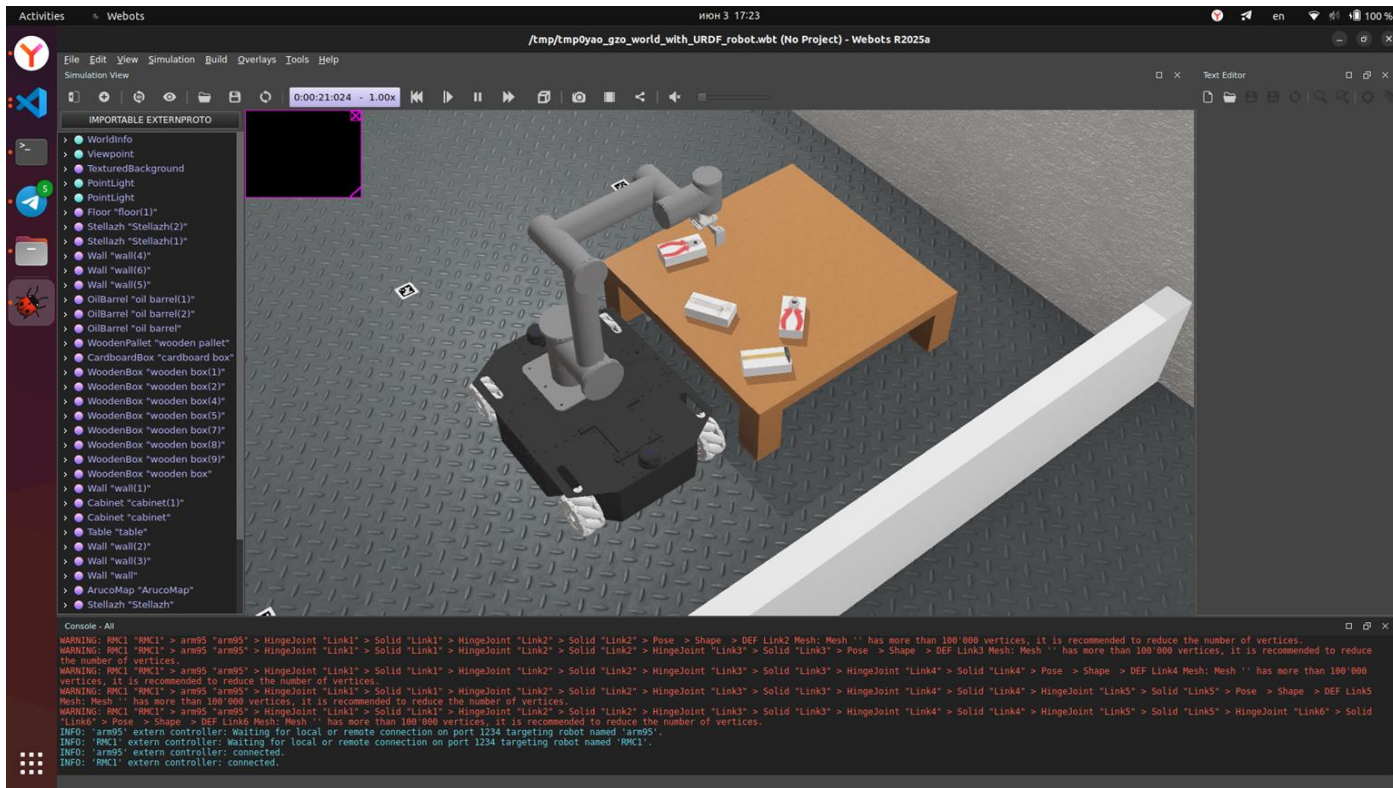
```
colcon build --symlink-install  
source /ros2_ws/install/setup.bash
```

После успешной сборки проекта запускаем готовую сцену в Webots командой

```
ros2 launch ar_webots_fms_ros2 module3.launch.py
```

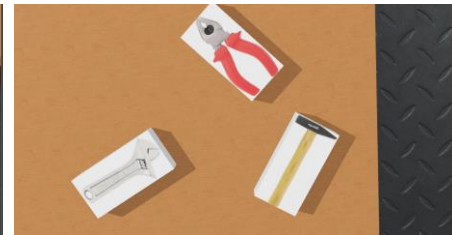
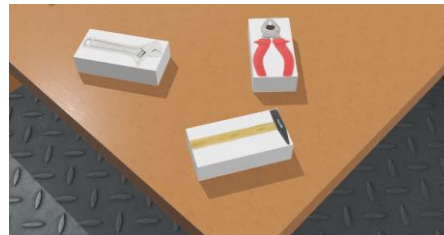
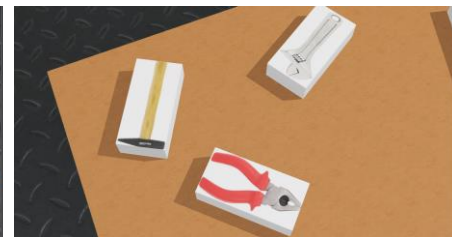
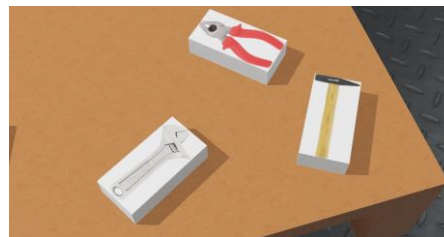
Продолжаем создание датасета

После выполнения команды должен открыться симулятор с нужной сценой



Продолжаем создание датасета

Теперь необходимо сделать N-ое количество скриншотов, на которых будут видны инструменты. Для каждого скриншота располагаем камеру под разными углами, чтобы инструменты на картинке были в различных положениях. Мы сделаем 11 скриншотов и искусственно разможем датасет, но вы можете сделать больше. Получим следующие картинки:



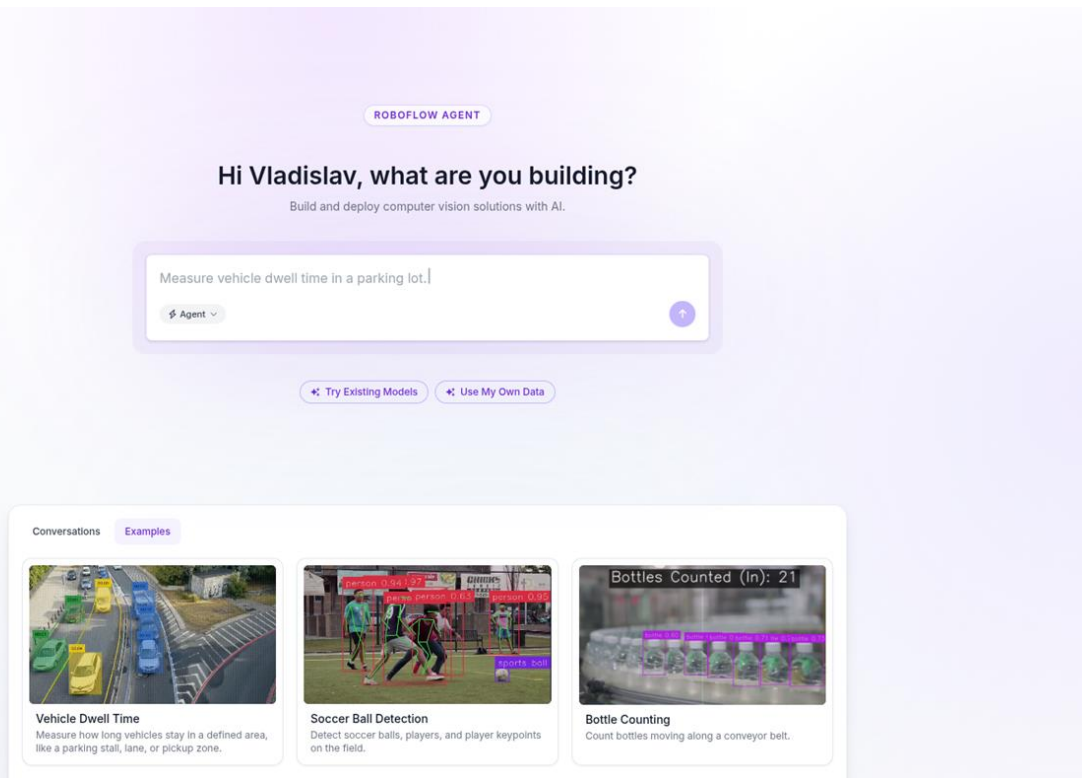
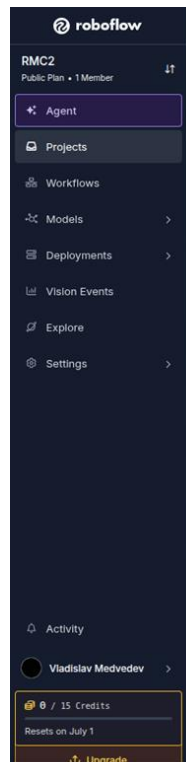
Переходим к разметке датасета

Заходим на сайт roboflow.com и регистрируемся. Здесь есть удобные инструменты для разметки датасета, его модификации и импорта под требуемую нейросеть.



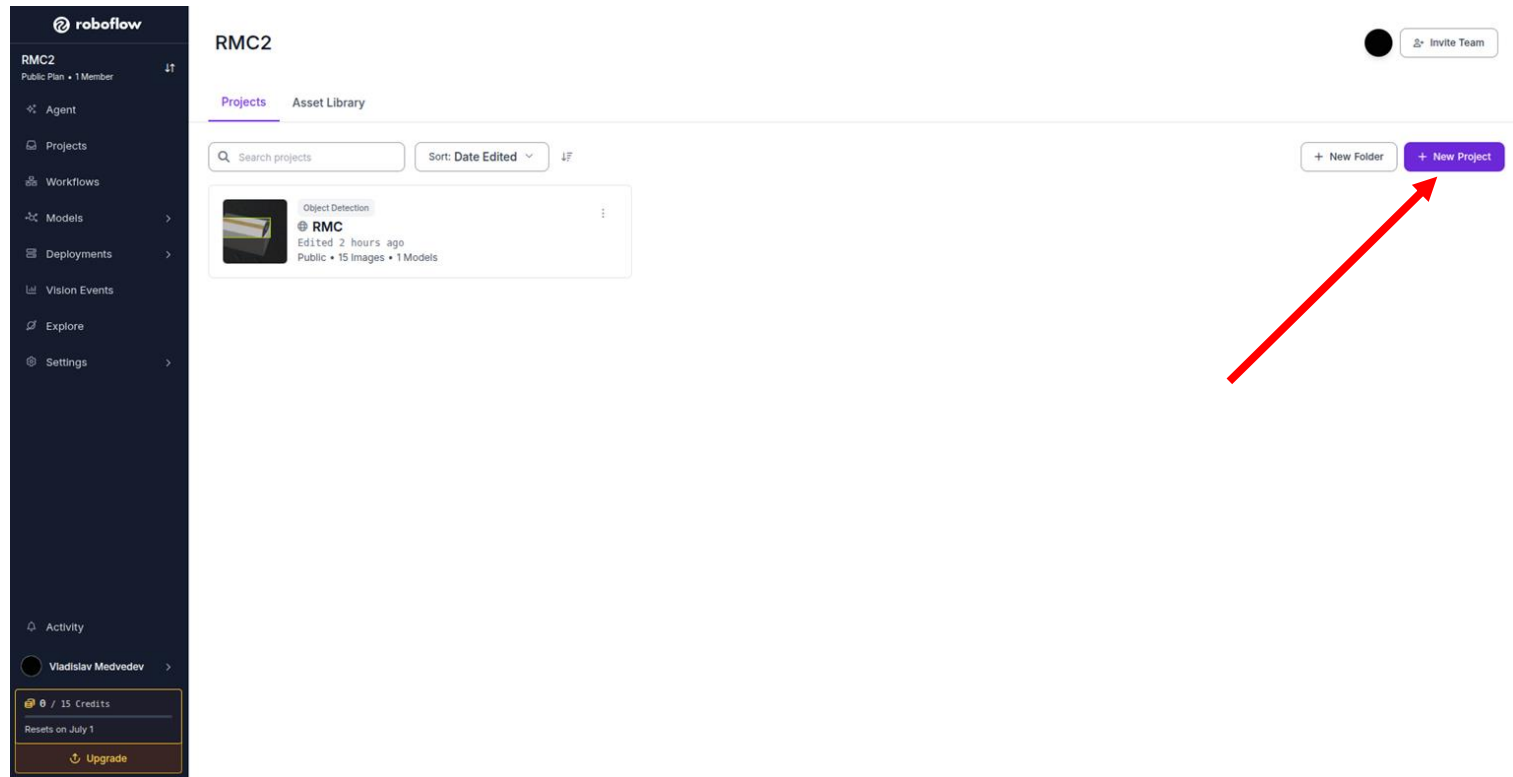
Разметка датасета

После регистрации на сайте вы должны попасть на следующую страницу, слева в меню нажимаем на Projects



Разметка датасета

Перейдя на страницу Projects, нажимаем кнопку для создания нового датасета



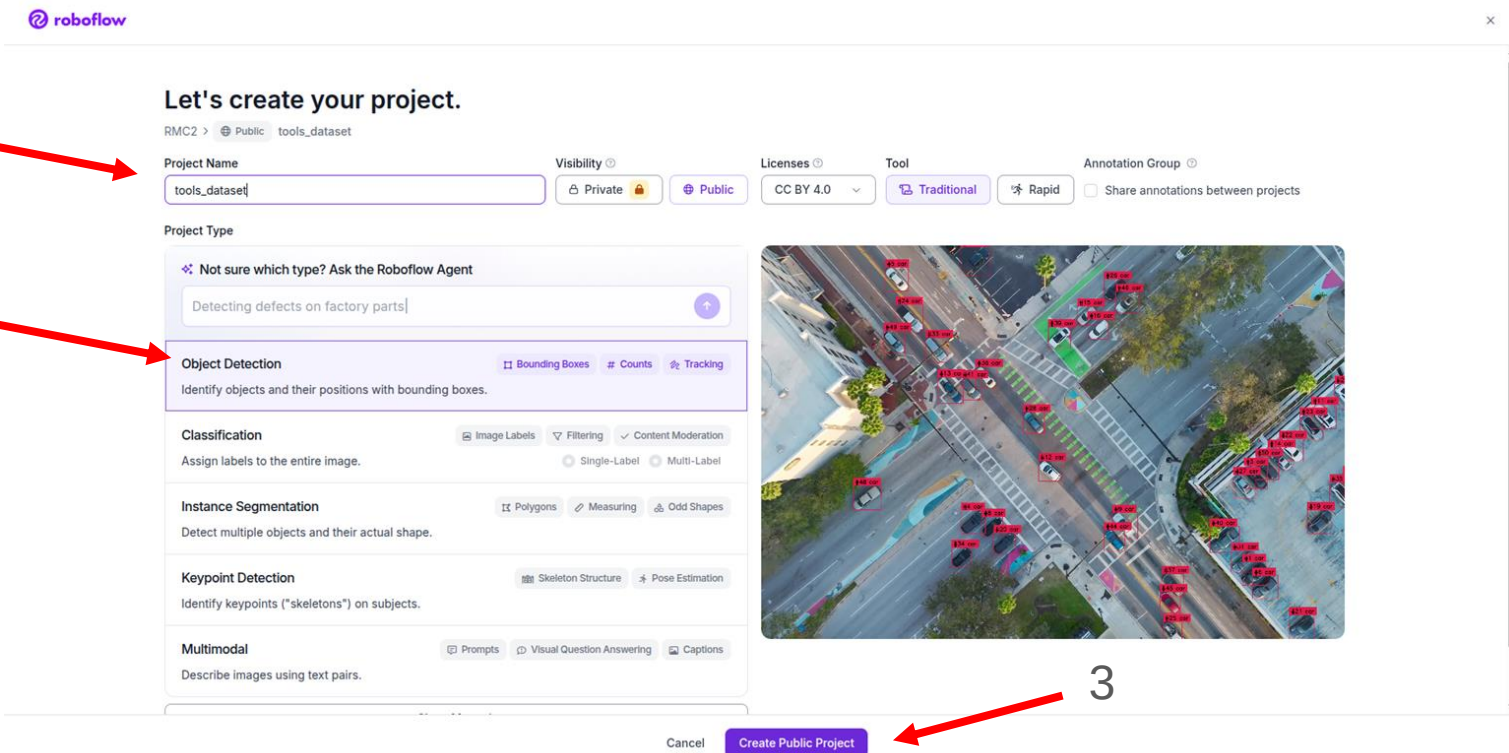
Разметка датасета

Попадаем на страницу создания проекта. Указываем название и выбираем тип “Object Detection”, создаем проект.

1

2

3



Let's create your project.

RMC2 > Public tools_dataset

Project Name: tools_dataset

Visibility: Private Public

Licenses: CC BY 4.0

Tool: Traditional Rapid

Annotation Group: Share annotations between projects

Project Type

Not sure which type? Ask the Roboflow Agent

Detecting defects on factory parts

Object Detection

Identify objects and their positions with bounding boxes.

Classification

Assign labels to the entire image.

Instance Segmentation

Detect multiple objects and their actual shape.

Keypoint Detection

Identify keypoints ("skeletons") on subjects.

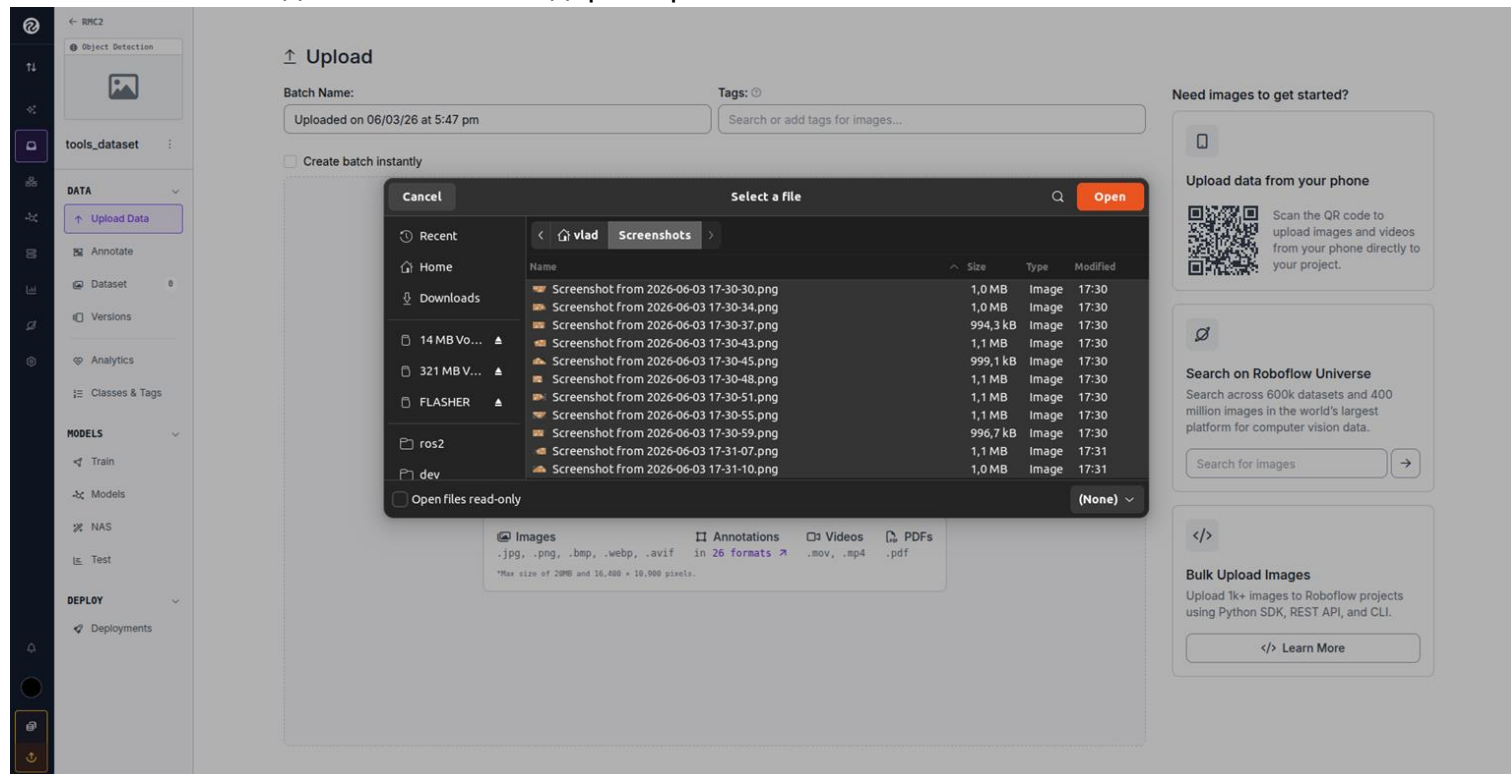
Multimodal

Describe images using text pairs.

Create Public Project

Разметка датасета

Проект откроется на странице загрузки изображений. Нажимаем Select Files и выбираем сделанные ранее скриншоты. На Ubuntu они должны лежать в директории Home/Screenshots



Разметка датасета

После загрузки изображений нажимаем “Save and Continue”

← RMC2

Object Detection

tools_dataset

DATA

Upload Data

Annotate

Dataset

Versions

Analytics

Classes & Tags

MODELS

Train

Models

NAS

Test

DEPLOY

Deployments

Upload

Batch Name:

Tags:

☐ Create batch instantly

All Images 11 Annotated 0 Not Annotated 11

Drag and drop images, annotations, and videos.

☐ .jpg, .png, .bmp, .webp, .avif ☐ in 26 formats ☐ .mov, .mp4

*Max size of 20MB and 16,400 × 10,900 pixels.

Screenshot from 2026-06-03 17:30-30.png

Screenshot from 2026-06-03 17:30-07.png

Screenshot from 2026-06-03 17:30-59.png

Screenshot from 2026-06-03 17:30-55.png

Screenshot from 2026-06-03 17:30-51.png

Screenshot from 2026-06-03 17:30-48.png

Screenshot from 2026-06-03 17:30-45.png

Screenshot from 2026-06-03 17:30-43.png

Screenshot from 2026-06-03 17:30-37.png

Screenshot from 2026-06-03 17:30-34.png

Screenshot from 2026-06-03 17:30-30.png

Want to add similar images? Powered by Objects365

0 selected + Add X

Разметка датасета

После того как наши изображения загрузятся на сайт, справа появятся кнопки. Нажимаем “Label Myself”, чтобы разметить датасет вручную.

The screenshot displays the Roboflow web application interface for dataset management. On the left is a dark sidebar with navigation icons and labels: DATA, MODELS, and DEPLOY. The main content area is titled 'Annotate' and 'Unassigned Images'. It features a search bar, filters for 'Split', 'Classes', 'Tags', and 'Sort By Newest', and a 'Show annotations' toggle. Below these are two rows of image thumbnails, each labeled 'Screenshot from 202...'. At the bottom of the main area, there's a selection bar showing '0 images selected' and buttons for 'Add Tags & Metadata', 'Reassign For Labeling', and 'Delete Image'. On the right side, a panel titled 'How do you want to label your images?' offers four options: 'Auto-Label Entire Batch' (Recommended), 'Label Myself' (highlighted with a red arrow), 'Label With My Team', and 'Hire Outsourced Labelers' (Upgrade).

Разметка датасета

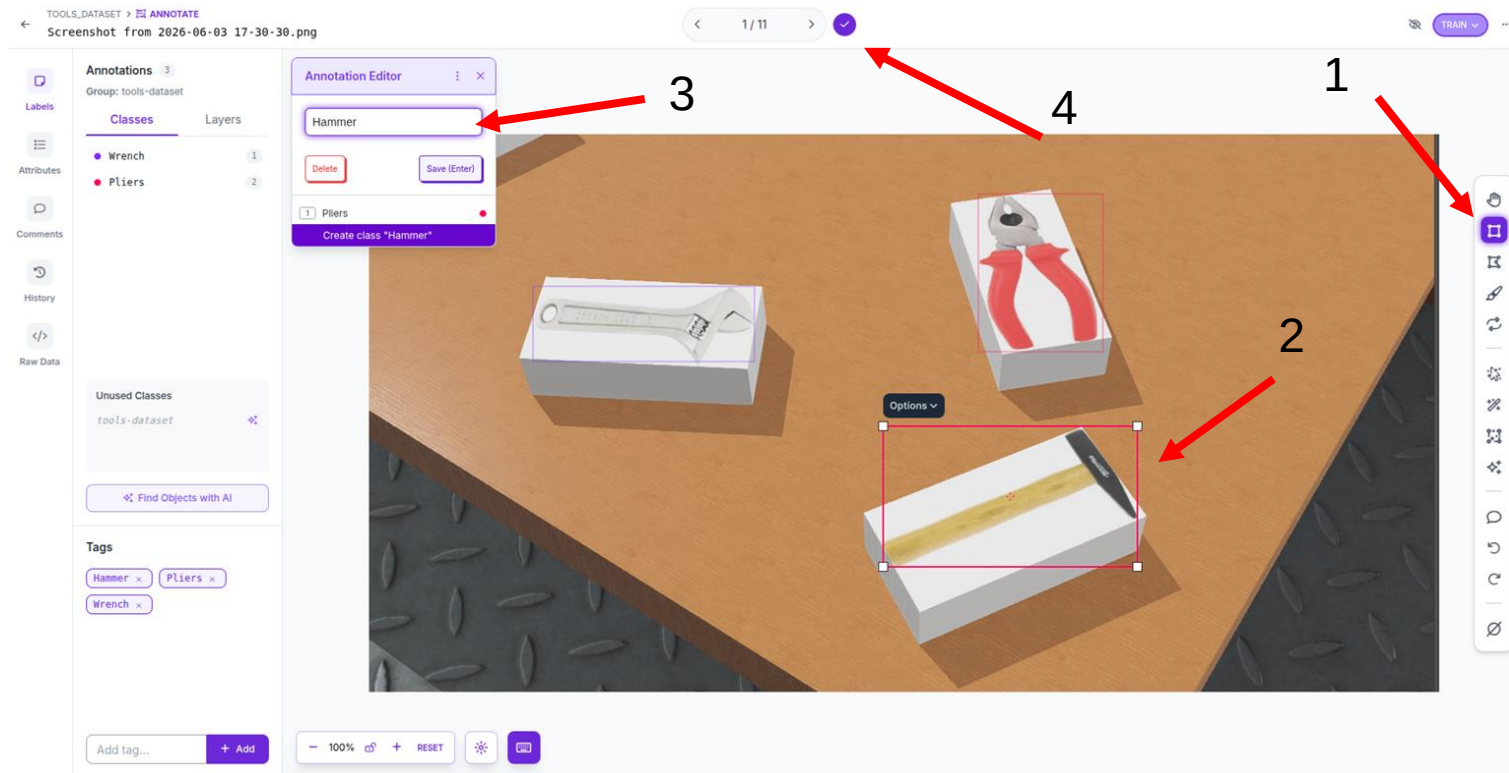
Подтверждаем начало разметки.

The screenshot displays the MosaicML Studio interface for a dataset named 'tools_dataset'. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for 'DATA' (Upload Data, Annotate, Dataset, Versions), 'MODELS' (Train, Models, NAS, Test), and 'DEPLOY' (Deployments).
- Top Bar:** Shows the upload date and time: 'Uploaded on 06/03/26 at 5:47 pm'. It also features a 'Start Annotating' button (highlighted with a red arrow) and a 'Submit for Review' button.
- Progress Section:** Indicates '11 Images' with '0 Annotated' and '11 Unannotated'.
- Instructions Section:** States 'No specific instructions were added when this job was assigned'.
- Assignment Section:** Shows the dataset is assigned to 'Vladislav Medvedev'.
- Timeline Section:** Displays a log entry: '\$Job created via API and assigned it to vladislav-medvedev.ru. 6/3/2026, 5:57:26 PM'.
- Image Grid:** A 2x6 grid of 12 screenshots showing various tools on a wooden surface. Each image has a small blue star icon in the bottom-left corner, indicating they are unannotated.

Разметка датасета

Откроется страница разметки. Выбираем справа инструмент “Bounding Box Tool” для прямоугольной обводки, обводим инструменты и прописываем их названия (Hammer, Wrench, Pliers), так мы добавим классы в список размечаемых.



Создание итоговой версии датасета

Размечаем на каждом изображении инструменты и завершаем процесс разметки. Откроется следующая страница. Теперь слева в меню надо перейти в раздел “Versions”, чтобы настроить наш датасет.

The screenshot displays the dataset management interface. On the left sidebar, the 'Versions' tab is highlighted with a red arrow. The main area shows a grid of 11 image thumbnails, each labeled 'Screenshot from 2026...'. Above the grid, there are search and filter options, including 'Search Images', 'Filter by filename', 'Split', 'Classes', 'Tags', 'Sort By Newest', and 'Search by Image'. A red arrow points to the 'Versions' tab in the sidebar. At the bottom, there is a status bar indicating '0 images selected' and various action buttons like 'Add Tags & Metadata', 'Reassign For Labeling', 'Mark Null', 'Change Dataset Split', and 'Delete Image'.

Создание итоговой версии датасета

Вводим имя версии и в разделе “Train/Test split” нажимаем “Edit”.

The screenshot displays the 'Versions' section of a dataset management tool. On the left is a sidebar with navigation options: DATA (Upload Data, Annotate, Dataset, Versions, Analytics, Classes & Tags), MODELS (Train, Models, NAS, Test), and DEPLOY (Deployments). The main area is titled 'Versions' and contains a 'Create New Version' button. Below this, a 'Version Name' field is filled with 'version1'. The 'Source Images' section shows 11 images and 3 classes. The 'Train/Test Split' section shows a training set of 11 images, a validation set of images, and a testing set of images. A red arrow points to the 'Edit' button next to the 'Train/Test Split' section. The 'Preprocessing' section is also visible, showing steps like 'Auto-Orient' and 'Resize'.

Versions

Create New Version Uses Credits

Prepare your images and data for training by compiling them into a version. Experiment with different configurations to achieve better training results.

Version Name:
version1

Source Images Images: 11 Classes: 3 Unannotated: 11 Edit

Train/Test Split Training Set: 11 images Validation Set: images Testing Set: images Edit

3 Preprocessing What can preprocessing do? Decrease training time and increase performance by applying image transformations to all images in this dataset.

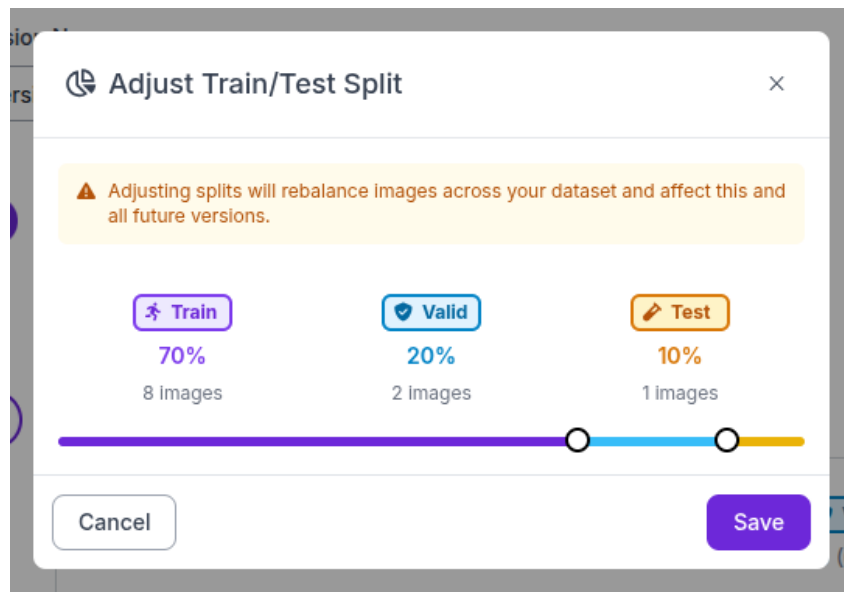
Auto-Orient Edit ×

Resize Stretch to 512×512 Edit ×

+ Add Preprocessing Step

Создание итоговой версии датасета

Нажимаем кнопку “Rebalance”, чтобы настроить разделение датасета. Необходимо разбить весь наш датасет на три категории данных. “Train” - данные, используемые для обучения. “Valid” - данные, которые модель использует для оценки процесса обучения. “Test” - данные, которые будут использоваться после окончания обучения, они не используются во время обучения вообще и с их помощью проверяется качество результатов обучения. Рекомендуется разделить данные в соотношении, указанном на картинке. Train - 70%, Valid - 20%, Test - 10%. После нажимаем “Save”.



Создание итоговой версии датасета

Переходим к разделу “Preprocessing”. Здесь можно настроить параметры предобработки изображений. Нажимаем “Edit”. И добавляем шаги “Auto-Orient” и “Resize” в режиме “Fit (black edges) in 512x512”. Нажимаем “Continue”.

The screenshot displays the 'Versions' section of a dataset management tool. On the left, a sidebar shows the navigation menu with 'Versions' selected. The main panel shows the 'Versions' tab with a message 'No versions created yet.' and a list of steps: 'Train/Test Split', 'Preprocessing', 'Augmentation', and 'Create'. The 'Preprocessing' step is highlighted with a red box and a red arrow pointing to it. The 'Preprocessing' step is configured with two steps: 'Auto-Orient' and 'Resize'. The 'Resize' step is configured with the mode 'Fit (black edges) in 512x512'. A red arrow points to the 'Edit' button next to the 'Preprocessing' step. Another red arrow points to the 'Continue' button at the bottom of the 'Preprocessing' configuration panel.

Versions

No versions created yet.

Classes: 3
Unannotated: 11

Train/Test Split [Edit](#)

Training Set: 8 images
Validation Set: 2 images
Testing Set: 1 images

3 Preprocessing [Edit](#)

Decrease training time and increase performance by applying image transformations to all images in this dataset.

Auto-Orient [Edit](#) ×

Resize [Edit](#) ×

Fit (black edges) in 512x512

+ Add Preprocessing Step

[Back](#) [Continue](#)

4 Augmentation [Edit](#)

5 Create [Edit](#)

Создание итоговой версии датасета

Переходим к разделу “Augmentation” и нажимаем “Add Augmentation Step”. Это нужно для размножения размера датасета.

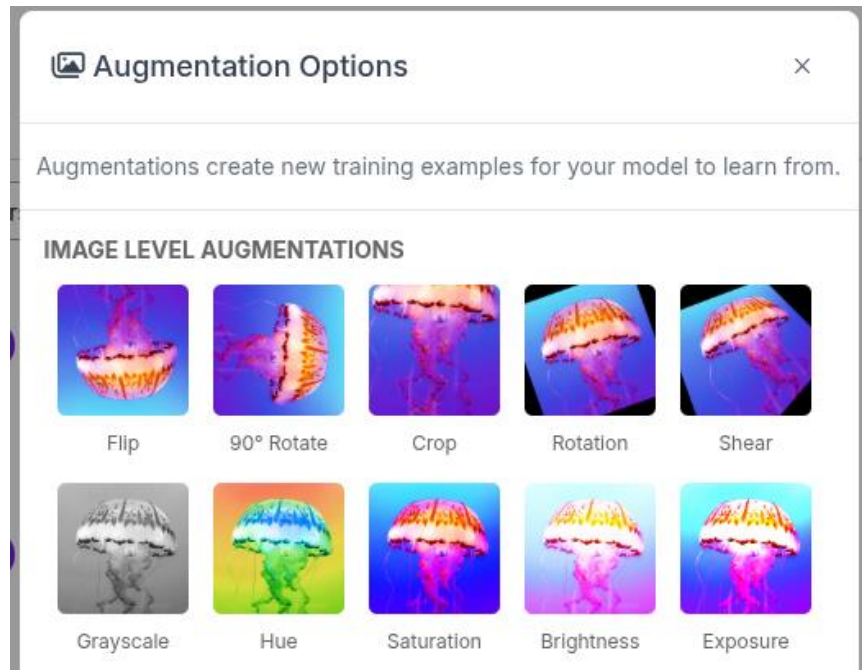
The screenshot displays the 'Versions' page in the ML Studio interface. The left sidebar shows the navigation menu with 'Versions' selected. The main panel shows the 'version1' configuration with the following steps:

- Source Images**: Images: 11, Classes: 3, Unannotated: 11. Edit
- Train/Test Split**: Training Set: 8 images, Validation Set: 2 images, Testing Set: 1 images. Edit
- Preprocessing**: Auto-Orient: Applied, Resize: Fit (black edges) in 512x512. Edit
- 4 Augmentation**: Create new training examples for your model to learn from by generating augmented versions of each image in your training set. Edit

In the 'Augmentation' step, there is a button labeled '+ Add Augmentation Step' which is highlighted by a red arrow. Below this button are 'Back' and 'Continue' buttons. At the bottom of the panel, there is a '5 Create' button.

Создание итоговой версии датасета

Откроется окно с различными способами модификации изображений. Так мы можем добавить несколько шагов обработки, автоматически отразив изображения по вертикали/горизонтали, повернуть изображения, обрезать изображения. В результате получим к исходным картинками еще модифицированные копии, увеличив размер датасета.



Создание итоговой версии датасета

Добавим несколько модификаций. Четких требований к модификаторам нет, мы рекомендуем выбрать те, что меняют размеры и ориентацию изображений. Изменять контрастность, цветовую гамму и т.п. не нужно.

На картинке показан список модификаторов, которые мы выбрали, и их настройки.

После добавления нужных вам модификаций, нажмите кнопку “Continue”.

Flip Horizontal, Vertical	Edit	×
90° Rotate Clockwise, Counter-Clockwise, Upside Down	Edit	×
Crop 0% Minimum Zoom, 29% Maximum Zoom	Edit	×
Rotation Between -15° and +15°	Edit	×
Shear ±15° Horizontal, ±15° Vertical	Edit	×
+ Add Augmentation Step		

[Back](#)[Continue](#)[Clear All](#)

Создание итоговой версии датасета

В последнем разделе “Create” выбираем максимальное доступное количество изображений для нашего бесплатного тарифа и нажимаем кнопку “Create”.

5

Create 

Review your selections and select a version size to create a moment-in-time snapshot of your dataset with the applied transformations.

Larger versions take longer to train but often result in better model performance. [See how this is calculated ↗](#)

Maximum Version Size

27 images (3x)

▼

Version Notes:

Add any version notes here...

Back

Create

Экспортируем датасет

На открывшейся странице нажимаем кнопку “Download Dataset”

The screenshot shows the Roboflow web interface for a dataset named 'tools_dataset'. The left sidebar contains navigation options: DATA (Upload Data, Annotate, Dataset, Versions), ANALYTICS, CLASSES & TAGS, MODELS (Train, Models, NAS, Test), and DEPLOY (Deployments). The main area displays the 'version1' dataset, generated on Jun 3, 2026 by Vladislav Medvedev. A red arrow points to the 'Download Dataset' button in the top right corner. Below the version information, there is a message: 'This version doesn't have a model.' with a 'Train Model' button and a 'How to Upload Custom Weights' link. At the bottom, the 'Dataset Split' section shows the distribution of images: TRAIN SET (24 Images, 89%), VALID SET (2 Images, 7%), and TEST SET (1 Images, 4%).

versions

version1
Generated on Jun 3, 2026 by Vladislav Medvedev

Download Dataset **Edit**

+ Create New Version

This version doesn't have a model.
Train an optimized, state of the art model with Roboflow or upload a custom trained model to use features like Label Assist and Model Evaluation and deployment options like our auto-scaling API and edge device support.

Train Model
Uses Roboflow Credits [View usage](#)

How to Upload Custom Weights

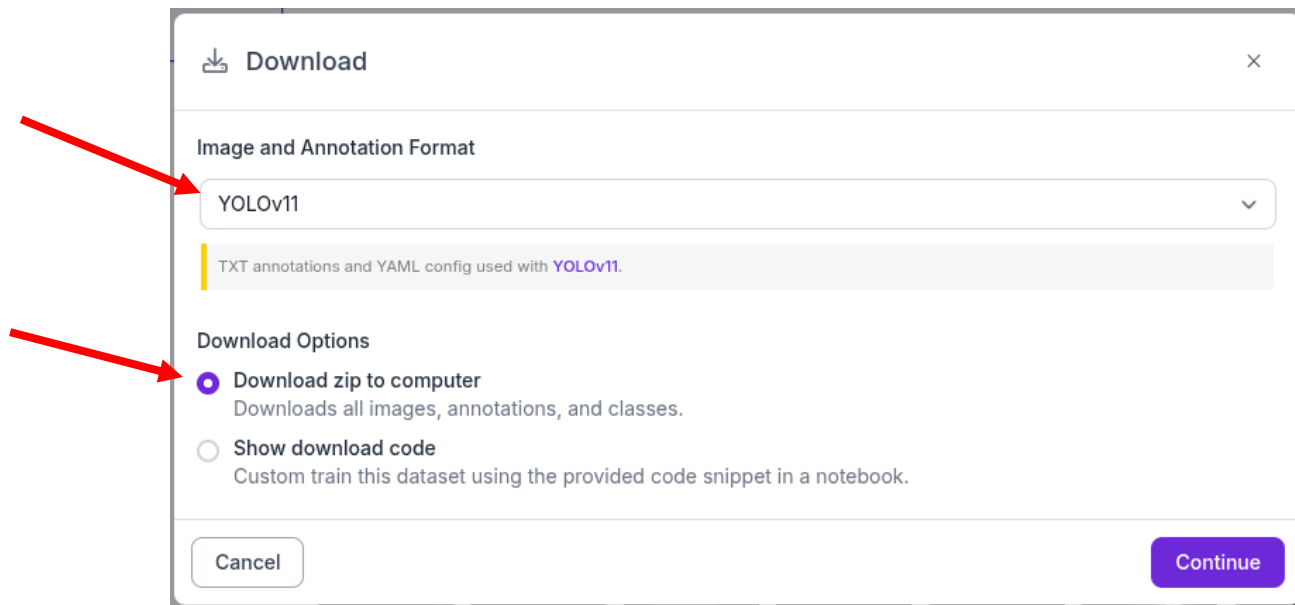
27 Total Images [View All Images](#)

Dataset Split

Set	Count	Percentage
TRAIN SET	24 Images	89%
VALID SET	2 Images	7%
TEST SET	1 Images	4%

Экспортируем датасет

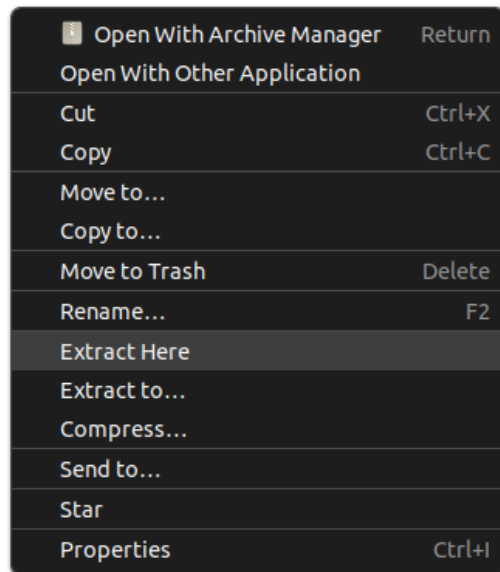
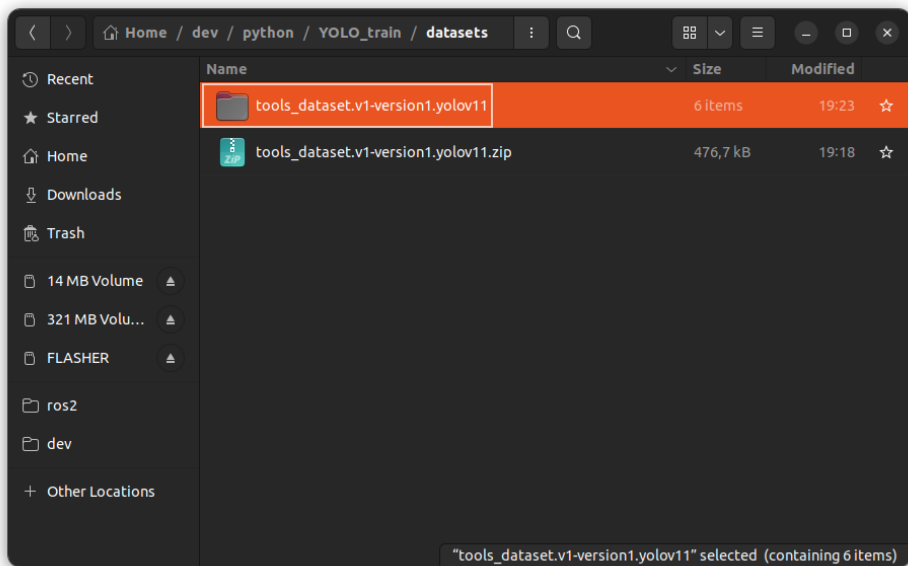
Откроется окошко, в котором нужно выбрать название нейросети, для обучения которой мы сохраняем датасет. Выберем YOLOv11. В параметрах “Download Options” выберем вариант “Download zip to computer”.



Подготовка к обучению модели

Итак, мы разметили и скачали датасет. Давайте подготовим данные к обучению. Можно создать отдельную директорию для этого. Назовем ее, например, “YOLO_train”.

В директории YOLO_train создаем директорию datasets и переносим туда скачанный архив. Нажимаем на него правой кнопкой мыши и разархивируем кнопкой “Extract Here”.



Создание виртуального окружения Python

Теперь, чтобы не засорять систему, создадим виртуальное окружение Python, в которое установим пакеты для работы с нейросетью YOLOv11. Открываем в терминал и переходим в директорию YOLO_train.

```
cd <путь до директории>/YOLO_train
```

Создаем виртуальное окружение venv командой

```
python -m venv venv
```

После этого необходимо активировать его командой

```
source venv/bin/activate
```

Если вы все правильно сделали и окружение активировалось, то перед приглашением к вводу в терминале появится надпись "(venv)"

```
yolo@vlad:~/ws$ python -m venv venv
yolo@vlad:~/ws$ source venv/bin/activate
(venv) yolo@vlad:~/ws$
```

Установка YOLO

Теперь установим пакет для работы с нейросетью YOLO. Предварительно активировав виртуальное окружение, пишем в терминале команду

```
pip install ultralytics
```

Ultralytics - это компания, которая разрабатывает YOLO. Установленный пакет будет содержать всё необходимое для обучения и инференса модели.

Установка может занять достаточно большое количество времени, в зависимости от скорости интернета, поэтому необходимо подождать.

После установки можно проверить, что все скачалось, введя в терминал команду

```
yolo
```

В ответ вам выведется список доступных команд и прочие подсказки, если все установлено.

```
(venv) yolo@vlad:~/ws$ yolo
Creating new Ultralytics Settings v0.0.6 file ✓
View Ultralytics Settings with 'yolo settings' or at '/home/yolo/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.

Arguments received: ['yolo']. Ultralytics 'yolo' commands use the following syntax:

  yolo TASK MODE ARGS

Where TASK (optional) is one of ['semantic', 'detect', 'pose', 'obb', 'classify', 'segment']
      MODE (required) is one of ['train', 'benchmark', 'predict', 'export', 'val', 'track']
      ARGS (optional) are any number of custom 'arg=value' pairs like 'imgsz=320' that override defaults.
      See all ARGS at https://docs.ultralytics.com/usage/cfg or with 'yolo cfg'

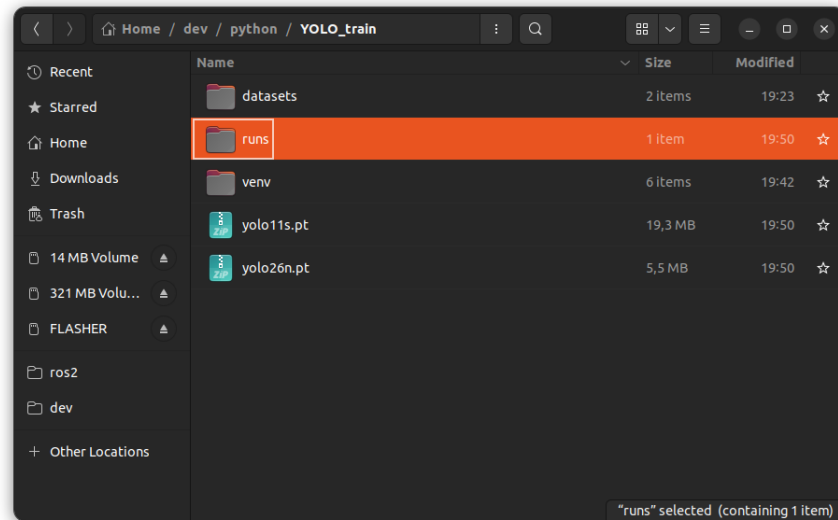
1. Train a detection model for 10 epochs with an initial learning_rate of 0.01
```

Обучение модели

Теперь можно запускать процесс обучения. Введите в терминале показанную ниже команду. Удостоверьтесь, что путь до файла data.yaml указан верно и совпадает с ВАШИМ именем директории. Файл data.yaml содержит всю информацию о датасете, который мы создали ранее.

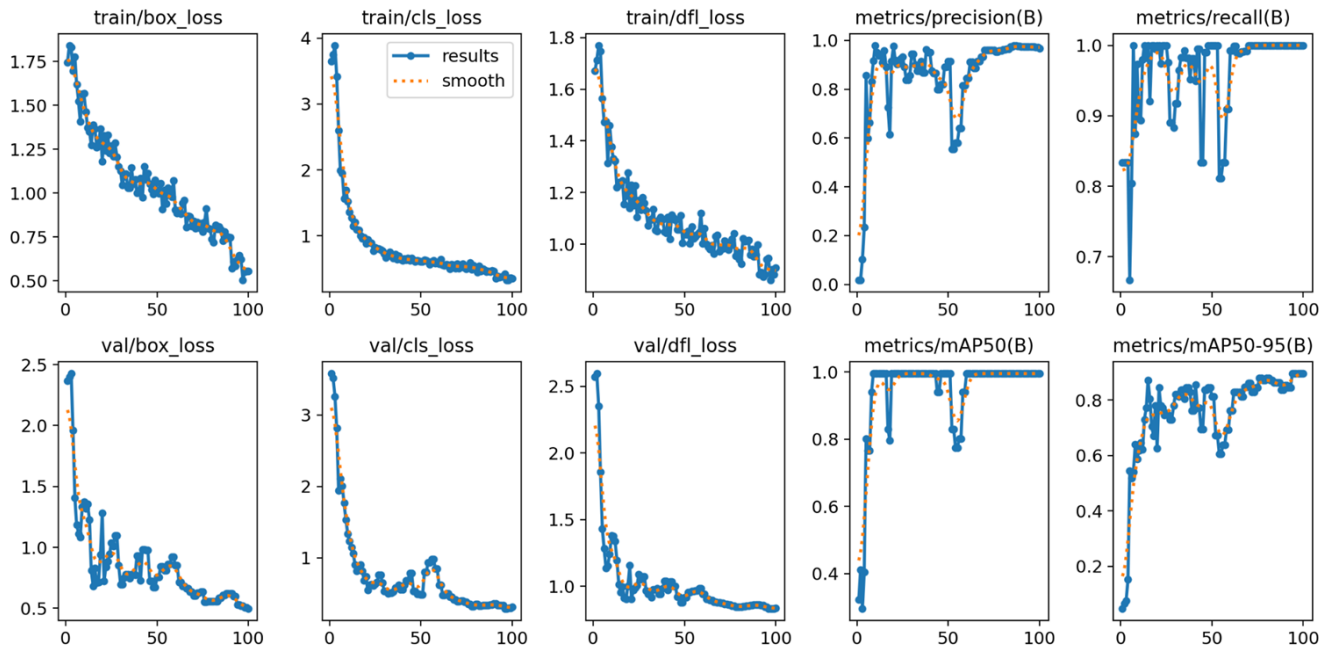
```
yolo task=detect \  
  mode=train \  
  model=yolo11s.pt \  
  data=datasets/tools_dataset.v1-version1.yolov11/data.yaml \  
  epochs=100 \  
  imgsz=512
```

Если у вас нет видеокарты, обучение нейросети может занять много времени. Поэтому, если захотите, то можете поэкспериментировать с длительностью обучения, изменив параметр epochs, поставив число поменьше. Помимо того, что вы потратите меньше времени на обучение, вы сможете проследить, насколько количество эпох влияет на качество обученной модели. В нашем случае при epochs=100 мы получили очень хорошие результаты. После завершения обучения появится директория runs, в которой лежат результаты обучения.



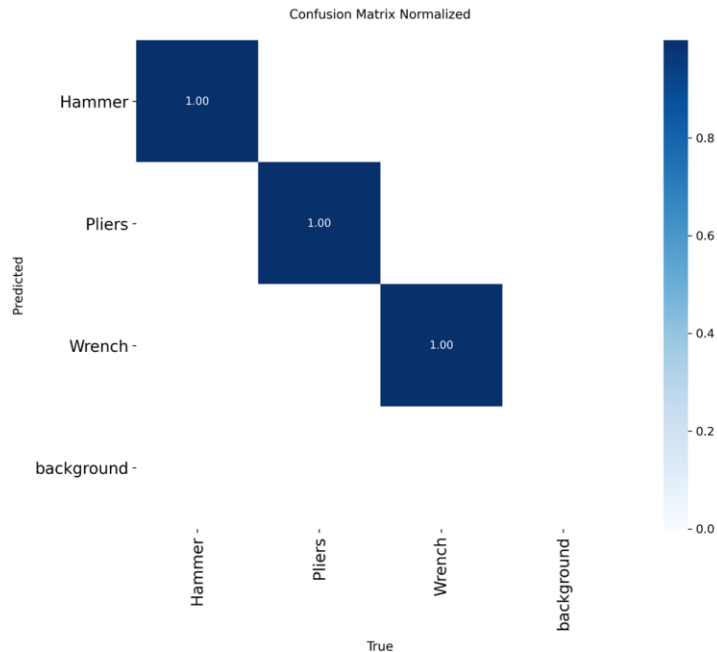
Результаты обучения

Давайте посмотрим на содержимое директории runs. Перейдем в нее, затем в detect и в train. Тут лежат изображения с графиками, которые показывают как менялись различные метрики на протяжении обучения. Посмотрим содержимое results.png. На этих графиках видно, как уменьшалась ошибка рисования обводки вокруг изображения, как уменьшалась ошибка классификации и прочие параметры.



Результаты обучения

Еще одной полезной метрикой является Confusion Matrix. Она показывает, к какому классу принадлежал объект из валидационного изображения и к какому классу его отнесла обученная нейросеть. Если обучение прошло успешно, эта матрица должна быть как можно ближе к диагональной матрице. Наличие числа в недиагональной ячейке означает, что нейросеть ошибочно распознала класс объекта и перепутала его с другим.



Обученная модель

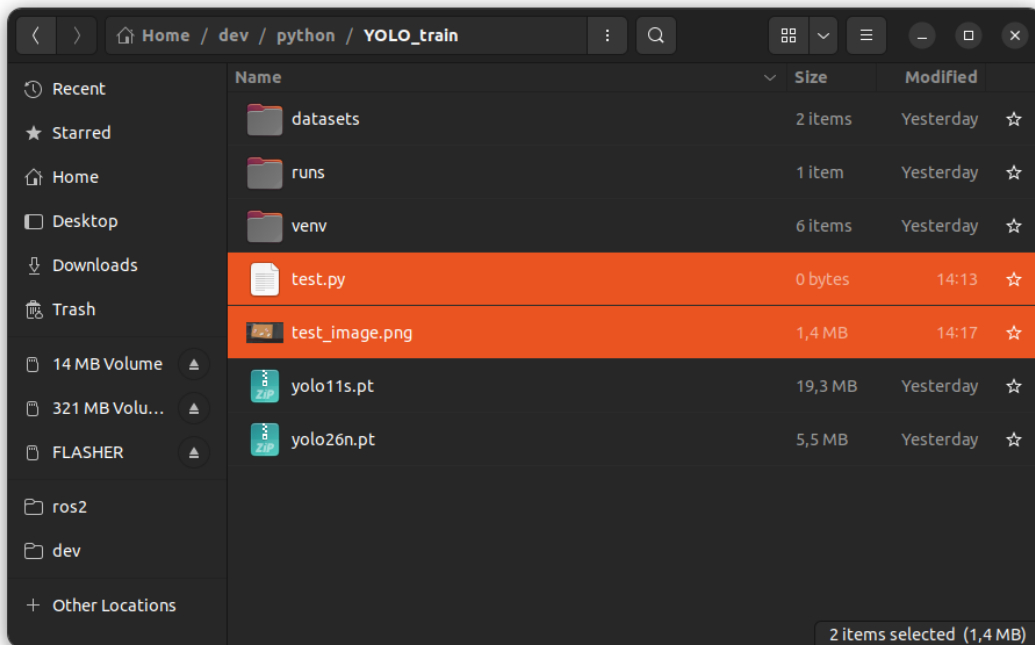
Веса обученной модели лежат в директории

YOLO_train/runs/detect/train/weights/[best.pt](#)

Именно эту модель необходимо использовать далее для поиска инструментов на изображении.

Запускаем обученную модель

Давайте теперь попробуем запустить модель, которую мы только что обучили. Создадим в директории YOLO_train файл [test.py](#) и в эту же директорию поместим еще один скриншот из сцены в Webots, который не использовался в обучении.



Запускаем обученную модель

Открываем [test.py](#) в удобном для вас редакторе и пишем небольшую программу.

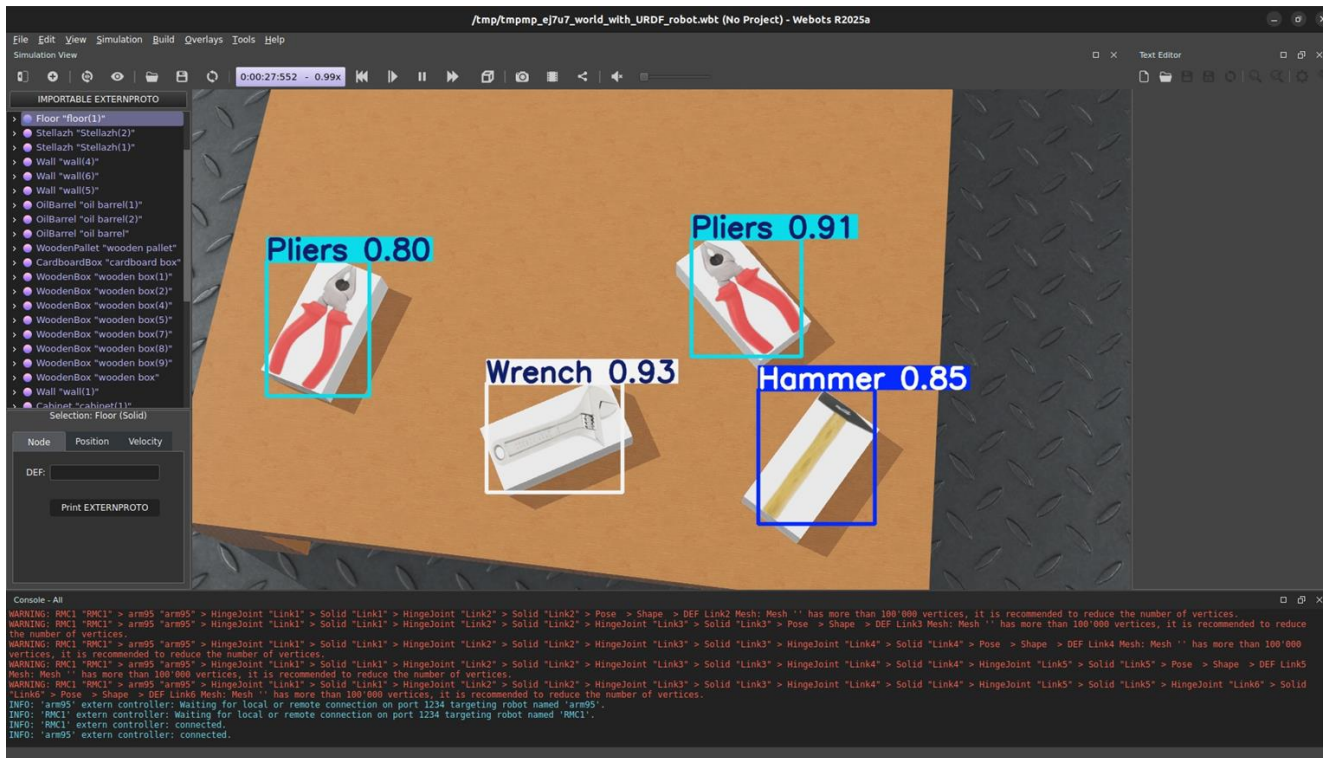
```
# Импортируем класс YOLO
from ultralytics import YOLO

# Открываем модель, которую обучили ранее
model = YOLO("runs/detect/train/weights/best.pt")

# Запускаем детектирование
results = model.predict(
    source='test_image.png', # Путь до картинки
    conf=0.5, # Отсеивать результаты ниже этого порога (0 >= conf >= 1)
    save=True # Сохранить результат
)
```

Запускаем обученную модель

Результат сохранится в YOLO_train/runs/detect/predict. Посмотрим, что получилось.



Запускаем обученную модель

Вы также можете использовать результаты детектирования в коде. Ниже представлен пример того, как получить размеры и расположение прямоугольников, которые обводят найденные объекты, уверенность модели в определении класса объекта и номер класса, который был определен

```
for result in results:
    # Проходим в цикле по каждому результату
    result.bboxes.xyxy      # Координаты левого верхнего и правого нижнего угла, (N, 4)
    result.bboxes.xywh      # Координаты левого верхнего угла, ширина и высота, (N, 4)
    result.bboxes.xyxy_norm  # То же, что xyxy, но координаты нормализованы, (N, 4)
    result.bboxes.xywh_norm  # То же, что xywh, но координаты нормализованы, (N, 4)
    result.bboxes.conf       # Уверенность в определении класса, (N, 1)
    result.bboxes.cls        # ID класса, (N, 1)
```